Reviewer Pack — Minimal Rank of Generalized Refined Invariants and Resolutio…

Entry 84ed15a8ed0e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 20:26:30 UTC

Minimal Rank of Generalized Refined Invariants and Resolution Proof Depth

Entry ID: `84ed15a8ed0e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 20:26:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Generalized Cohomology) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['Let F be a CNF formula with n variables. Define the generalized refined invariant ρ(F) as the rank of the generalized cohomology group associated with the set of clauses in F. Then, for all CNFs F, ρ(F) = Ω(log t*(F)) where t*(F) is the Resolution proof depth.', 'For a counterexample, provide a CNF F such that ρ(F) < log t*(F).', 'ρ is computable in polynomial time with respect to the size of the input CNF.']

Rationale (proposer's reasoning):

> ['Generalized cohomology provides a rich algebraic structure to study invariants of geometric objects. If the minimal rank of these invariants can be shown to be related to the complexity of resolution proofs, it might expose a deeper connection between algebraic topology and computational complexity.', 'Such a relationship could potentially lead to new techniques for proving lower bounds on resolution proof complexity.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e36586dccfcd33ca`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF F with n variables, if the correlation between ρ(F) and log t*(F) exceeds 0.8 with p-value < 0.05 across 30 random seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"generalized cohomology" AND "Resolution proof complexity"` - `"algebraic topology" AND "polynomial time computation" AND "CNF formula"` - `"minimal rank" IN "generalized refined invariants" AND "resolution proof depth"`

Top relevant hits considered: - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2208.01392v2] The Analytic Minimal Rank Sard Conjecture - [http://arxiv.org/abs/2406.15089v2] Sentences over Random Groups II: Sentences of Minimal Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def rank(matrix):
        m, n = len(matrix), len(matrix[0])
        for i in range(m):
            if matrix[i][i] == 0:
                return -1  # Singular matrix
            for j in range(i + 1, m):
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]
        rank = sum(1 for row in matrix if any(x != 0 for x in row))
        return rank

    def resolution_depth(clauses):
        # Simplified version of Resolution depth calculation
        return len(clauses) + random.randint(1, 5)

    n = random.choice([5, 10, 15, 20, 30, 40])
    clauses = [[random.randint(-n, n) for _ in range(n)] for _ in range(n)]
    rho = rank([[1 if abs(x) == i+1 else 0 for x in clause] for clause in clauses])
    t_star = resolution_depth(clauses)

    if rho < 0:
```

```
        return {
            "metric_name": "rho",
            "metric_value": rho,
            "instances_tested": n,
            "conjecture_holds": False,
            "counterexample": "singular_matrix"
        }

    metric_value = rho
    conjecture_holds = metric_value >= math.log(t_star)
    counterexample = "" if conjecture_holds else f"rho={rho}, log t*={math.log(t_star)}"

    return {
        "metric_name": "rho",
        "metric_value": metric_value,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a698536f.py", line 69, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a698536f.py", line 39, in run_trial
    rho = rank([[1 if abs(x) == i+1 else 0 for x in clause] for clause in clauses])
```

```
                                    ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed due to an undefined variable 'i', which prevents us from verifying the conjecture's claim. The critic has challenged the support condition, and since the test did not complete successfully, we cannot confirm the support for the conjecture. | next: Review the test code for errors and ensure that all variables are correctly defined before rerunning the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11823
2	preregistration	ollama_remote	glm4:latest	0	0	5250
3	novelty	ollama_remote	glm4:latest	0	0	4700
4	novelty	ollama_remote	glm4:latest	0	0	7378
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13581
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10777
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10063
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8698
9	judge	ollama_remote	glm4:latest	0	0	8314

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 80584 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/84ed15a8ed0e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/84ed15a8ed0e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/84ed15a8ed0e.tar.gz` (if generated)